School of Computer Science, UPES, Dehradun

A Project Report on

Autonomous Valet Parking Agent

Submitted for the course

CSAI4014 Algorithms for Intelligent Systems and Robotics

B.Tech CSE (AI & ML), Semester VII

Project No.: P13
Group No.: G14
Batch: B2

Submitted by

S. No.	Name	SAP ID
1	Saurabh Kumar	500124288
2	Naman Chanana	500119485
3	Anirvan Ranjan	500119330
4	Aditya Pratap Singh Sisodia	500121985
5	Priyal Sharma	500122041

Submitted to

Course Faculty, CSAI4014
School of Computer Science, UPES, Dehradun

Academic Year 2026-2027 | October 2026

Declaration

We declare that this project report titled “Autonomous Valet Parking Agent” is our own work, carried out for the course CSAI4014 Algorithms for Intelligent Systems and Robotics. The code, results and text are original, and all sources used are cited in the References. The source code is available in our group GitHub repository, and every member’s work can be traced to commits from his or her own account.

Name	**SAP ID**	**Signature**
Saurabh Kumar	500124288	
Naman Chanana	500119485	
Anirvan Ranjan	500119330	
Aditya Pratap Singh Sisodia	500121985	
Priyal Sharma	500122041	

Date: _____ Place: Dehradun

Index

Declaration . . . . . **i**

List of Figures . . . . . **iv**

List of Tables . . . . . **iv**

Writing Allocation . . . . . **v**

Abstract . . . . . **vi**

1 Introduction **1**
- 1.1 Background 1
- 1.2 Why it matters in real life and in industry 1
- 1.3 What makes the problem hard 1
- 1.4 What this report presents 2

2 Problem Statement and Objectives **2**

3 Literature and Industry Review **3**
- 3.1 Path planning for car-like robots 3
- 3.2 Path tracking 3
- 3.3 Agent architectures 3
- 3.4 Industry products and standards 4

4 Novelty **4**

5 Syllabus Mapping and Theory **5**
- 5.1 Syllabus mapping 5
- 5.2 Kinematic bicycle (Ackermann) model 5
- 5.3 A* and Hybrid A* 6
- 5.4 Path-tracking control 7
- 5.5 Time to collision and Kalman filter 7

6 System Design **8**
- 6.1 Block diagram 8
- 6.2 Three-layer agent architecture 8
- 6.3 Robot and environment model 9
- 6.4 Sensors and actuators 10
- 6.5 Software architecture 10

7 Methodology and Algorithms **10**
- 7.1 Mission state machine and control cycle 10
- 7.2 Declarative layer: slot selection 11
- 7.3 Declarative layer: Hybrid A* with goal funnel 12
- 7.4 Procedural layer: path execution 13
- 7.5 Reflexive layer 13
- 7.6 Localisation 14

8 Simulation Setup **14**
- 8.1 Platforms 14
- 8.2 World and parameters 15
- 8.3 How to run the code 15

9 Results and Analysis **16**
- 9.1 A complete mission 16
- 9.2 E1: Deliberative planners 16

9.3 E2: Steering laws and speed loop 19
9.4 E3: Reflexive layer and pedestrian safety 19
9.5 E4–E7: Agent decisions, noise and robustness 20
9.6 Parking accuracy, and what did not work 22
10 Contribution **23**
11 Co-learning Reflection **24**
12 Conclusion and Future Scope **25**
References **26**
Appendix: Links and Logs **27**

List of Figures

1 Kinematic bicycle model of the Ackermann car (reference point: rear-axle centre) . . . 6
2 Block diagram of the valet parking system . . . 8
3 Declarative–procedural–reflexive hierarchy of the agent (solid: commands, dashed: events) . . . 8
4 Parking-lot environment (one random scenario: 70% occupied, two pedestrian crossings) . . . 9
5 Sensor layout: 7 range sensors on each bumper and 6 on each side, mounted 0.5 m high . . . 10
6 Software architecture: one agent package shared by Webots and the headless twin . . . 11
7 Mission finite-state machine of the valet agent . . . 11
8 Webots world: BMW X5 valet car, parked cars and a pedestrian (seed 3, 70% occupancy) . . . 15
9 A complete mission: planned path, driven path and car footprints (blue forward, orange reverse) . . . 16
10 Speed, steering angle and cross-track error during the mission of Fig. 9 . . . 17
11 (a) Grid A* and Hybrid A* routes to slot B3. (b) Search trees with and without the goal funnel . . . 17
12 E1 — success rate, planning time and node expansions of the three planners 18
13 Speed loop: (a) unit-step response, (b) Bode plot of the open loop $L(s) = C(s)G(s)$ . . . 19
14 Same lot and pedestrians: speed and obstacle distance seen by the fixed-distance and TTC reflexes . . . 20
15 E5 — success rate and final lateral error against GPS noise . . . 21
16 (a) Success rate against lot occupancy (E6). (b) Valet cycle length for the two slot policies (E4) . . . 22

List of Tables

1 Comparison of existing approaches and the gap addressed . . . 4
2 AISR syllabus topics used in the project . . . 6
3 Vehicle parameters (Webots BmwX5) . . . 9
4 Simulation parameters . . . 15
5 E1 — planner comparison (30 lots, 70% occupancy) . . . 18
6 E2 — path-tracking laws (30 lots, Hybrid A* paths) . . . 18
7 E3 — reflex designs with four crossing pedestrians (30 lots) . . . 19
8 E4 — slot-selection policy (30 lots, 85% occupancy) . . . 20
9 E5 — GPS noise: raw GPS vs Kalman-filtered pose (15 lots per level) . . . 21
10 E6 / E7 — lot occupancy and the “slot taken” event (15 lots each) . . . 21
11 Technical contribution of each member . . . 23

Writing Allocation

The writing work was divided equally among the five members. Each member also wrote his or her own row of Section 10 and own reflection in Section 11.

Member	Name and SAP ID	Sections written	Pages
M1	Saurabh Kumar (500124288)	Abstract, 1, 2, 13 (References)	5
M2	Naman Chanana (500119485)	3, 4	5
M3	Anirvan Ranjan (500119330)	5, 6	5
M4	Aditya Pratap Singh Sisodia (500121985)	7, 8	5
M5	Priyal Sharma (500122041)	9, 12, Appendix	5
All	Every member	10 (own row), 11 (own reflection), Declaration	–

Abstract

Automated valet parking (AVP) lets a driver leave the car at a drop-off zone while the car finds a free slot and parks itself. This project builds an AVP agent for a car-like (Ackermann) vehicle in the Webots simulator, organised as a three-layer declarative–procedural–reflexive hierarchy. The declarative layer chooses a slot with a utility function and plans a kinematically feasible route with Hybrid A*. The procedural layer follows the route forward and in reverse with a Pure Pursuit steering law and a PID speed loop. The reflexive layer runs every 48 ms and overrides the others for emergency stops, pedestrian safety and final docking. The same Python agent runs the Webots car and a headless twin, which we used for 525 Monte Carlo missions in random lots. Hybrid A* parked successfully in 100% of missions, against 0% for grid A*. Our novelties are a slot "goal funnel" that cuts node expansions 5.5 times, a swept-footprint time-to-collision reflex (0% contacts against 17% for a fixed-distance stop), a utility-based slot choice that shortens the valet cycle by 7.6%, and a GPS–odometry Kalman filter that keeps success at 100% under 0.3 m GPS noise.

Keywords: automated valet parking, Hybrid A*, Ackermann steering, pure pursuit, reactive layer, intelligent agent, Webots.

1. Introduction

1.1 Background

Parking is one of the most tiring parts of city driving. Drivers circle a crowded lot to find a free slot and then make several low-speed manoeuvres in a tight space, where most minor collisions with other cars, pillars and pedestrians happen. *Automated valet parking* (AVP) removes the driver from this task: the driver leaves the car at a drop-off zone at the entrance, and the car drives itself to a free slot, parks, and later drives itself to a pick-up zone when the owner calls it [1], [2]. AVP is one of the first driverless functions to reach the road, because it works at low speed (below about 10 km/h) in a closed, mapped area, which makes it far easier to make safe than open-road driving [3].

1.2 Why it matters in real life and in industry

- **Industry deployments.** Bosch and Mercedes-Benz received approval in Germany in 2022 for a driverless (SAE Level 4) AVP service in a multi-storey car park at Stuttgart airport. The car park's own sensors guide the car to a free slot [2]. ISO 23374-1 now standardises such systems, including the split of tasks between the vehicle and the parking facility [1].
- **Space and cost.** Driverless cars need no space to open their doors and can park more tightly, so a car park can hold more cars.
- **Safety.** Low-speed scrapes in car parks are one of the most frequent kinds of insurance claim. A system that keeps a safe distance to people and objects at all times directly reduces them.
- **Fleets and logistics.** The same technology moves new cars in factory yards and ports, and parks and charges robotaxis and delivery vans without a driver.

1.3 What makes the problem hard

A car is not a point robot. It cannot turn on the spot, it has a minimum turning radius set by its steering limit, and it must often reverse (Fig. 9 in Section 9). A path that ignores these constraints, such as the shortest path on a grid, may be impossible to drive, so the planner must reason about position *and* heading [4]. The car must also react within a fraction of a second to things that are not on any map, such as a person walking between parked cars. One planner cannot do both jobs well. This is why the AISR syllabus presents layered architectures, which combine slow deliberation with fast reflexes [5], [6].

1.4 What this report presents

We built a complete valet parking agent for the BMW X5 Ackermann car of the Webots vehicle library [7]. The agent chooses a slot, plans a drivable route, drives it forward and in reverse, and parks nose-out (reverse-in) so that retrieval is quick and safe. A reflexive safety layer protects pedestrians. Section 2 defines the problem and objectives. Section 3 reviews the literature and industry. Section 4 states our novelty, and Sections 5–8 give the theory, design, algorithms and simulation setup. Section 9 presents quantitative results from 525 simulated missions, Section 10 the contribution of each member, Section 11 our co-learning reflections, and Section 12 the conclusions.

2. Problem Statement and Objectives

Problem statement. A car-like vehicle with Ackermann steering starts at the drop-off point of a 39.6 m $\times$ 36 m parking lot with 48 perpendicular slots in four rows. 50–92% of the slots are already occupied, and pedestrians may cross the aisles. The vehicle must (i) choose a free slot, (ii) plan a collision-free route that respects its turning radius, using a *deliberative* planner, (iii) drive the route and reverse into the slot, using *reactive* rules for the close-range manoeuvre and for safety, and (iv) stop inside the slot within ± 0.3 m laterally and $\pm 5^\circ$ in heading, with no contact with any car, wall or person.

Measurable objectives.

1. **O1 Success rate.** Park successfully in at least 95% of random lots at 70% occupancy, with zero collisions.
2. **O2 Accuracy.** Final lateral error below 10 cm and heading error below 2° on average.
3. **O3 Real-time planning.** Plan a full route in under 5 s on a laptop CPU in Python.
4. **O4 Pedestrian safety.** With four pedestrians crossing the aisles, reach zero contacts while still completing the mission.
5. **O5 Comparison.** Compare at least two methods for each layer: two planners, two steering laws, three reflex designs and two slot-selection policies.

Assumptions. The ground is flat and the lot map (walls, slot lines) is known. As in infrastructure-supported AVP [1], [2], the parking facility reports which slots are occupied and the outline of each parked car. Parked cars do not move, but another car may take a slot while our car is on its way. The car's position comes from GPS (noise $\sigma = 0.05$–0.3 m), its heading from an inertial unit and its speed from wheel odometry. The car carries 26 short-range distance sensors.

Limits. We use a kinematic (not dynamic) vehicle model in the planner, low speeds (at most 2.2 m/s forward and 0.9 m/s reverse), one parking style (perpendicular, reverse-in), and

simple scripted pedestrians.

3. Literature and Industry Review

3.1 Path planning for car-like robots

Graph search. A* finds the shortest path on a graph using an admissible heuristic [8]. On an occupancy grid it is fast, but it treats the car as a point or disc and ignores heading, so its paths contain sharp corners that a car cannot follow.

Kinematically feasible curves. Reeds and Shepp showed that the shortest path for a car that can drive forward and backward with a bounded turning radius is made of at most five arcs and straight lines [9]. These curves are exact, but they ignore obstacles.

Hybrid A*. Dolgov et al. combined both ideas in the planner of Stanford's "Junior" car (DARPA Urban Challenge) [4]. The search runs over a grid of (x, y, θ) cells, but each node stores a continuous pose reached by simulating the car's motion, so every path is drivable. Hybrid A* is now the standard planner for parking in research and industry, and it is part of open-source driving stacks such as Autoware and Apollo [10].

Sampling-based planners. RRT and RRT* also handle car kinematics. However, their paths are random and need smoothing, and their run time varies widely, which is undesirable for a repeatable parking manoeuvre [10].

3.2 Path tracking

Pure Pursuit steers the rear axle along a circular arc towards a "look-ahead" point on the path [11]. It is simple, stable and works in reverse. The Stanley controller of Stanford's DARPA Grand Challenge car corrects heading and cross-track error at the front axle [12]. It is very accurate forward but was not designed for reversing. Snider compared both and found Pure Pursuit more robust at low speed [13].

3.3 Agent architectures

Brooks' subsumption architecture stacks simple reflexive behaviours, where a higher-priority behaviour *subsumes* (overrides) a lower one [5]. Gat's three-layer architecture separates a slow deliberator, a sequencer that executes plans, and a fast reactive controller [6]. This is the declarative–procedural–reflexive hierarchy of the AISR syllabus. Russell and Norvig classify agents as reflex, model-based, goal-based and utility-based [14]. Our slot choice is a utility-based agent.

3.4 Industry products and standards

- **Bosch / Mercedes-Benz Intelligent Park Pilot** (Stuttgart P6, 2022): infrastructure cameras locate free slots and guide the car; the car executes the manoeuvre without a driver (SAE Level 4) [2].
- **Driver-assist park systems** (e.g. Valeo Park4U, Tesla Autopark): the car finds a slot with its ultrasonic sensors and steers itself into it, but a driver must supervise (SAE Level 2) [3].
- **ISO 23374-1:2023** defines the AVP system framework, the vehicle–facility interface, and the requirement that the vehicle must stop for any obstacle in its path [1].
- **Time-to-collision (TTC)** is the classic safety measure for collision warning [15]. Most park-assist systems still use fixed distance thresholds.

Table 1: Comparison of existing approaches and the gap addressed

Approach	Strength	Limitation
Grid A* [8]	Fast, optimal on the grid	Ignores heading and turning radius; path may not be drivable
Reeds–Shepp curves [9]	Exact shortest car paths	Ignore obstacles
Hybrid A* [4]	Drivable, obstacle-aware	Slow to hit a narrow goal such as a slot (many node expansions)
Pure Pursuit [11] / Stanley [12]	Simple, well studied	Stanley is weak in reverse; neither handles new obstacles
Subsumption / 3-layer [5], [6]	Fast reflexes + planning	Reflex rules are usually fixed-distance stops
Industry AVP [1], [2]	Proven, certified	Closed source; relies on expensive infrastructure

What is missing. Table 1 shows three gaps that we address. (1) Hybrid A* wastes most of its search time trying to hit the exact parking pose. (2) Common reflex rules check a straight corridor ahead of the bumper, so they miss what the *side* of a turning car sweeps through. (3) Textbook examples pick the nearest free slot and ignore the second half of the valet task, which is driving the car back out.

4. Novelty

Compared with the textbook examples and the systems reviewed in Section 3, our work adds the following. Each point is supported by our own results in Section 9.

1. **Slot "goal funnel" for Hybrid A*.** Instead of searching until a node lands within a few centimetres and degrees of the parked pose, the search stops at any pose on the slot's centre line, 2.5–7 m out, that is aligned within 0.2 m and 6°. From there a collision-checked straight reverse finishes the path. This turns a tiny goal region into a long, thin one. *Evidence:* node expansions fell from 54,470 to 9,968 (5.5× fewer), mean planning time fell from 27.5 s to 5.1 s, and success rose from 80% to 100% (Table 5, Fig. 11).

2. **Swept-footprint time-to-collision reflex.** The reflex layer moves the car's outline along the arc of the *current* steering angle and finds the first sensed point it would touch. It then limits speed so that the time to reach that point is at least 2.5 s. This catches side-swipes during turns and ignores parked cars that are close but not on the swept path. *Evidence:* with four pedestrians, contacts fell from 20% (no reflex) and 17% (fixed-distance stop) to 0%, and success rose to 100% (Table 7).

3. **Utility-based slot choice for the whole valet cycle.** The agent ranks slots by drive-in distance + drive-out distance to the pick-up zone + a manoeuvre-difficulty penalty, instead of taking the nearest slot. *Evidence:* the mean valet cycle shortened from 54.9 m to 50.7 m (7.6%), and success went from 97% to 100% (Table 8).

4. **One agent, two simulators, with event handling.** The same Python agent drives the Webots car and a headless twin. The twin allowed 525 reproducible Monte Carlo missions. Its mission state machine re-selects a slot when another car takes it (100% success, Table 10). A light GPS–odometry Kalman filter keeps success at 100% under 0.3 m GPS noise, where raw GPS gives 20% (Table 9).

5. Syllabus Mapping and Theory

5.1 Syllabus mapping

Table 2 lists the AISR syllabus topics used and where each appears in the project.

5.2 Kinematic bicycle (Ackermann) model

An Ackermann car steers both front wheels so that all wheels roll about one centre of rotation on the rear-axle line. It can be modelled as a "bicycle" with one front and one rear wheel (Fig. 1) [16]. With the rear-axle centre (x, y) as the reference point, heading θ, speed v (negative when reversing), steering angle δ and wheelbase L:

$$\dot{x} = v\cos\theta, \qquad \dot{y} = v\sin\theta, \qquad \dot{\theta} = \frac{v}{L}\tan\delta, \qquad |\delta| \le \delta_{max}. \tag{1}$$

Table 2: AISR syllabus topics used in the project

Syllabus topic	Where it is used
Declarative–Procedural–Reflexive hierarchy	The three layers of `agent.py`: planning, path execution, reflex rules (Sections 6, 7)
Intelligent agents	PEAS description; utility-based slot selection; mission state machine (Section 7.1)
Path planning / Planning to move	Grid A*, Hybrid A* with motion primitives, Dijkstra heuristic, goal funnel (Section 7.2)
Moving (mobile ground robots)	Kinematic bicycle (Ackermann) model, turning radius, reverse driving (Section 5.2)
Open/closed-loop control	PID speed loop, Pure Pursuit and Stanley steering feedback, speed profile (Sections 5.4, 7.3)
Time and frequency domain, stability	Step response and Bode plot of the speed loop, phase margin (Section 9.3)
Robotic perception	26 range sensors, ray casting, GPS/odometry Kalman filter (Sections 6.3, 7.5)
Planning uncertain movements	Noise in GPS, heading and range; Monte Carlo evaluation (Section 9)
Software architectures	Shared agent package for Webots and twin; layered modules (Section 6.4)

The path curvature is $\kappa = \tan\delta / L$, so the minimum turning radius is $R_{min} = L/\tan\delta_{max}$. For our car, $L = 2.995$ m and $\delta_{max} = 0.55$ rad give $R_{min} = 4.87$ m, which is why the car cannot turn into a 2.8 m wide slot from a 7 m aisle in one forward move. The rear-axle reference is also the origin of Webots car models, so no offset is needed.

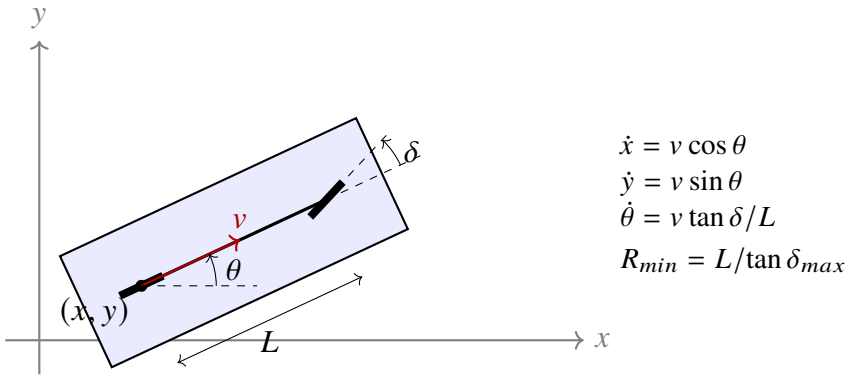


Figure 1: Kinematic bicycle model of the Ackermann car (reference point: rear-axle centre)

5.3 A* and Hybrid A*

A* expands nodes in order of $f(n) = g(n) + h(n)$, where g is the cost from the start and h an estimate of the remaining cost. If h never overestimates (it is *admissible*), the first path

found to the goal is optimal [8]. Hybrid A* [4] keeps the same priority queue, but a node is a continuous pose (x, y, θ), and its children are produced by integrating Eq. (1) for a fixed arc length with a set of steering angles, forward and in reverse (motion primitives). A closed set over discretised cells ($0.5\ \text{m} \times 0.5\ \text{m} \times 5^\circ$) prevents re-expanding similar poses. Our step cost is

$$c = \ell\,(1 + 0.6\,[\text{reverse}]) + 0.4\,|\delta| + 0.8\,|\delta - \delta_{prev}| + 6\,[\text{gear change}], \tag{2}$$

where $\ell = 1$ m is the primitive length and $[\cdot]$ is 1 when the condition holds. The heuristic is $h = \max(d_{Euclid}, d_{Dijkstra})$, where $d_{Dijkstra}$ is the obstacle-aware shortest distance on a 0.5 m grid, computed once from the goal.

5.4 Path-tracking control

Pure Pursuit [11] chooses a look-ahead point at distance L_d on the path and steers along the circular arc through it. If α is the angle between the direction of motion and the look-ahead point,

$$\kappa = \frac{2\sin\alpha}{L_d}, \qquad \delta = \sigma \arctan(L\kappa), \qquad L_d = \max(1.6, 0.8|v|), \tag{3}$$

where $\sigma = +1$ forward and $\sigma = -1$ in reverse. In reverse, the direction of motion is $\theta + \pi$ and $\dot\theta$ changes sign with v in Eq. (1).

Stanley [12] uses the heading error ψ_e and the cross-track error e of the front axle:

$$\delta = \psi_e + \arctan\frac{k\,e}{k_s + |v|}, \qquad k = 1.2,\ k_s = 0.5. \tag{4}$$

PID speed control. The speed loop computes an acceleration command

$$a_{cmd}(t) = K_p e_v(t) + K_i \int_0^t e_v\,d\tau - K_d \frac{dv}{dt}, \qquad e_v = v_{ref} - v, \tag{5}$$

with $K_p = 1.8$, $K_i = 0.6$, $K_d = 0.05$, saturation to $[-3, 1.5]$ m/s^2 and anti-windup. The drivetrain is modelled as a first-order lag $\tau\dot a = a_{cmd} - a$ with $\tau = 0.45$ s, so the plant from a_{cmd} to v is $G(s) = 1/(s(\tau s + 1))$.

5.5 Time to collision and Kalman filter

Time to collision is the time left before contact if nothing changes: $\text{TTC} = d/v_{close}$ [15]. Our reflex rule turns it around and limits the speed so that $\text{TTC} \geq T_{safe}$ (Eq. (7) in Section 7.4). For localisation, a Kalman filter predicts the position from odometry ($x_k = x_{k-1} + v\,\Delta t\cos\theta$, variance $P \leftarrow P + Q\,\Delta t$) and corrects it with GPS ($K = P/(P + R)$, $x \leftarrow x + K(z - x)$, $P \leftarrow (1 - K)P$) [17].

6. System Design

6.1 Block diagram

Figure 2 shows the full system. The parking facility (infrastructure) reports the free slots and the outlines of parked cars, as in ISO 23374 [1]. The car's own sensors feed a Kalman filter and the reflex layer. The three agent layers turn this information into a steering angle and a speed command for the Webots `Driver` (vehicle library).

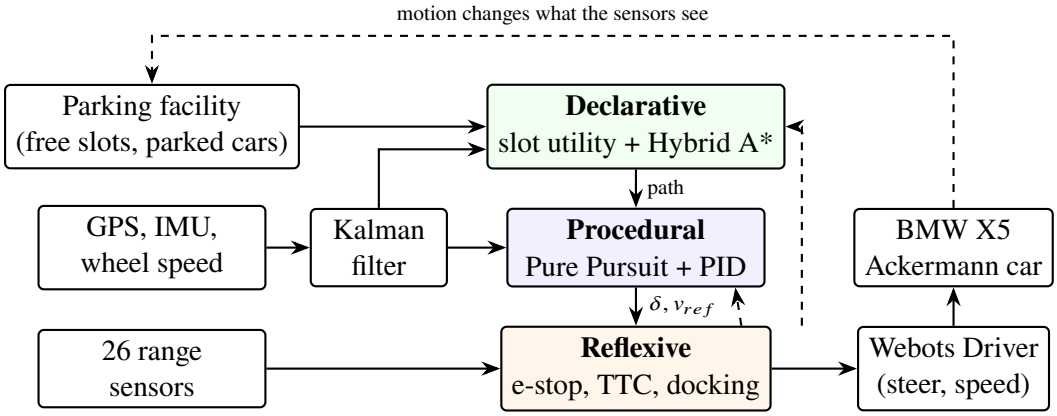


Figure 2: Block diagram of the valet parking system (dashed: reflex override and "blocked → re-plan" events)

6.2 Three-layer agent architecture

Figure 3 shows the agent's internal structure. The layers differ in how often they run and how far ahead they look. A higher-priority (lower) layer can override the command of the layer above it, as in subsumption [5].

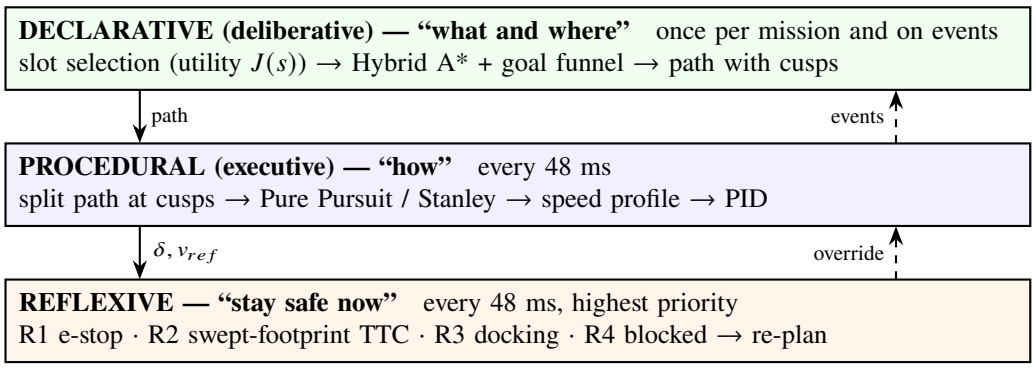


Figure 3: Declarative–procedural–reflexive hierarchy of the agent (solid: commands, dashed: events)

PEAS description of the agent [14]. *Performance:* parked inside a free slot, accuracy, time, no contact. *Environment:* parking lot, parked cars, pedestrians, other cars taking slots. *Actuators:* steering angle, speed (or throttle/brake and gear). *Sensors:* GPS, IMU, wheel speed, 26 range sensors, facility messages.

6.3 Robot and environment model

Environment. The lot (Fig. 4) has four rows of 12 slots (2.8 m × 5.5 m), two 7 m aisles, a 6 m entry lane on the left, a low kerb between the back-to-back middle rows, and walls on all sides. The drop-off (start) point is at the bottom left and the pick-up zone at the top left. Parked cars are boxes of random size (4.3–4.9 m × 1.75–1.95 m), placed with random offsets of up to 0.2 m and 3°. Pedestrians walk across the aisles at 0.9–1.4 m/s.

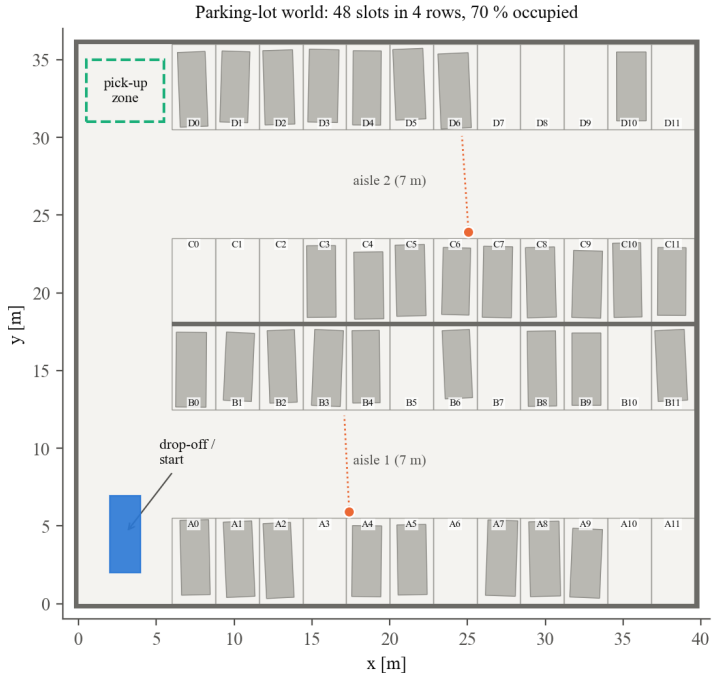


Figure 4: Parking-lot environment (one random scenario: 70% occupied, two pedestrian crossings)

Vehicle. Table 3 lists the vehicle parameters. They match the Webots `BmwX5` model.

Table 3: Vehicle parameters (Webots BmwX5)

Parameter	**Value**	**Note**
Wheelbase L	2.995 m	BmwX5 PROTO
Length × width	4.945 m × 1.95 m	overhangs 0.95 / 1.00 m
Max steering δ_{max} / rate	0.55 rad / 1.2 rad/s	planner uses 88%
Min turning radius R_{min}	4.87 m	$L/\tan\delta_{max}$
Speed limit forward / reverse	2.2 / 0.9 m/s	AVP speeds < 10 km/h
Drivetrain lag τ, max accel	0.45 s, 1.5 m/s^2	first-order model

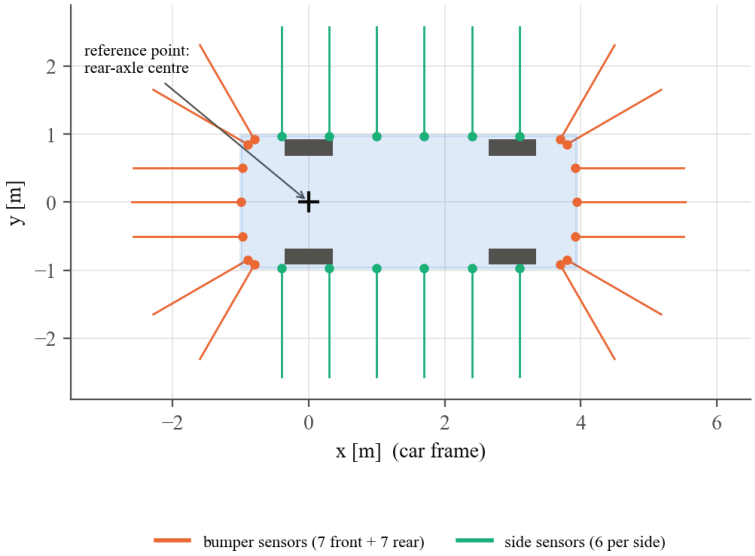


Figure 5: Sensor layout: 7 range sensors on each bumper and 6 on each side, mounted 0.5 m high

6.4 Sensors and actuators

Figure 5 shows the 26 range sensors (Webots `DistanceSensor`, range 8 m, noise 2 cm), modelled on the 12–16 ultrasonic sensors of production park-assist systems. Each bumper has a centre sensor, two straight sensors at ±0.5 m and corner sensors at ±30° and ±60°, so a 0.7 m wide pedestrian cannot hide between two rays. Six side sensors per side, spaced 0.7 m, cover the flanks of the car during a turn. A GPS (accuracy 5 cm) and an inertial unit (yaw) sit at the rear-axle centre. The actuators are the steering angle and the cruising speed of the Webots `Driver` (or throttle, brake and gear in the optional torque mode).

6.5 Software architecture

Figure 6 shows the code modules. The important design decision is that `valet/` is pure Python with no Webots imports. It is called by a thin Webots controller *and* by the headless twin, so every algorithm is tested in both, and the Webots world is generated from the same lot model, so the two can never drift apart.

7. Methodology and Algorithms

7.1 Mission state machine and control cycle

The agent is driven by the finite-state machine of Fig. 7. At every control step (Δt = 48 ms in Webots, 50 ms in the twin) it: (1) reads GPS, IMU, wheel speed and the 26 ranges; (2) updates the Kalman filter; (3) runs the declarative layer if the state is SELECT or PLAN; (4) runs the procedural layer to get (δ, v_{ref}); (5) lets the reflex layer modify (δ, v_{ref}); (6) closes

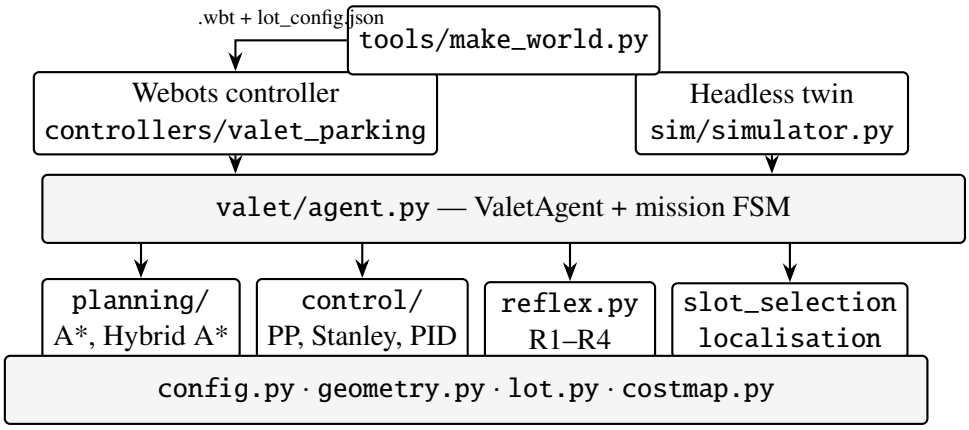


Figure 6: Software architecture: one agent package shared by Webots and the headless twin

the speed loop with the PID and sends the commands; and (7) logs the data.

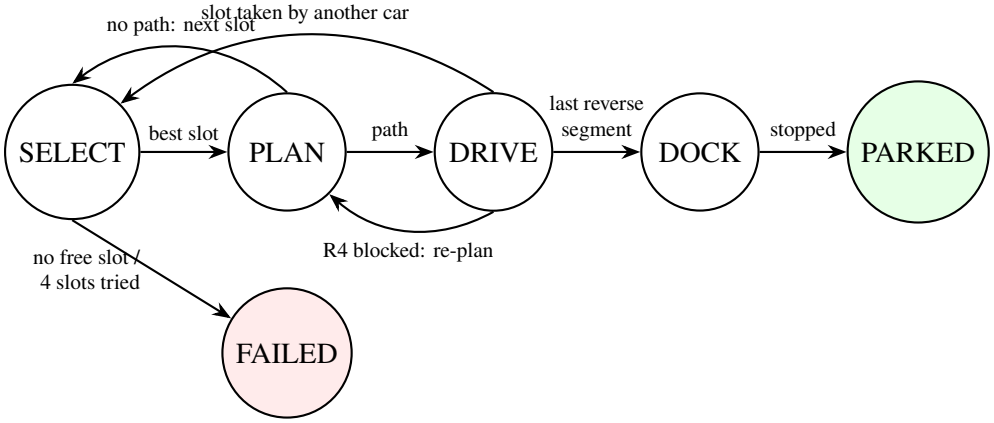


Figure 7: Mission finite-state machine of the valet agent

7.2 Declarative layer: slot selection

The agent evaluates every free slot s with the cost (negative utility)

$$J(s) = D_{in}(s) + w_{out}\, D_{out}(s) + w_m\, M(s), \qquad w_{out} = 1,\ w_m = 4\ \text{m}, \tag{6}$$

where D_{in} is the driving distance from the car to the slot, D_{out} the driving distance from the slot to the pick-up zone, and $M(s)$ the manoeuvre difficulty: the number of occupied neighbours (0–2), plus 1 if the slot across the aisle is occupied (less room to swing the front), plus 1 for a slot at the end wall. D_{in} and D_{out} for *all* slots come from two Dijkstra runs on the inflated grid, so ranking all 48 slots takes only a fraction of a second. The baseline policy takes the slot nearest to the car.

7.3 Declarative layer: Hybrid A* with goal funnel

Algorithm 1 gives our Hybrid A*. A pose is collision-free if every point of the car outline (sampled every 0.2 m) is more than the safety margin (0.2 m) from the nearest obstacle. The distance comes from a Euclidean distance transform of a 0.1 m occupancy grid, so each check is a few array look-ups. Primitives use 88% of the steering limit, which leaves a reserve for the feedback controller (Section 9.6 explains why this matters). The goal is the reverse-in pose: rear bumper 0.4 m from the slot's back line, nose facing the aisle.

Algorithm 1 Hybrid A* with slot goal funnel

Input: start pose q_s, parking pose q_g, costmap C

Output: drivable path with direction labels (forward / reverse)

$h_{map} \leftarrow$ Dijkstra from q_g on the inflated grid ▷ obstacle-aware heuristic
OPEN $\leftarrow \{q_s\}$ with $f = h(q_s)$; CLOSED $\leftarrow \emptyset$
while OPEN not empty and expansions $< N_{max}$ **do**
    $n \leftarrow$ pop node with lowest f; skip it if cell(n) $\in$ CLOSED; add cell(n) to CLOSED
    if FUNNEL(n, q_g) **then** ▷ novelty 1
        return path(n) + straight reverse from n to q_g
    end if
    for each direction $d \in \{+1, -1\}$, steering $\delta \in \{-0.88\delta_{max}, \ldots, +0.88\delta_{max}\}$ **do**
        $q' \leftarrow$ integrate Eq. (1) for 1 m in 0.25 m steps
        if all intermediate poses are free in C **then**
            $g' \leftarrow g(n)$+ step cost Eq. (2); push q' with $f = g' + \max(h_{map}, d_{Euclid})$ if g' improves its cell
        end if
    end for
end while
return failure ▷ agent tries the next slot

function FUNNEL(n, q_g)
    express n in the slot frame: distance along the axis a, lateral offset e, heading error $\Delta\theta$
    return $2.5 \le a \le 7$ m **and** $|e| < 0.2$ m **and** $|\Delta\theta| < 6°$ **and** straight segment to q_g is free
end function

The baseline **grid A*** runs 8-connected A* (octile heuristic, no corner cutting) on a 0.25 m grid whose obstacles are inflated by half the car width plus the margin. It then shortens the path by line-of-sight smoothing. It drives into the slot nose-first, because it has no notion of heading or reversing.

7.4 Procedural layer: path execution

1. **Segmenting.** The path is split at every cusp (change of gear) into constant-direction segments.
2. **Steering.** For the active segment, the nearest waypoint is found in a moving window, and the steering angle comes from Pure Pursuit (Eq. 3) or Stanley (Eq. 4). For Stanley in reverse we use the rear axle as reference and add a curvature feed-forward $\delta_{ff} = -\arctan(L\kappa)$. Without it, Stanley needs a large steady error to hold an arc.
3. **Speed profile.** $v_{ref} = \min\big(v_{max},\ \sqrt{1/\kappa_{ahead}},\ \sqrt{2a_b(d_{end} - 0.5|v|)} + 0.1\big)$, with the cruise speed 2.2 m/s forward and 0.9 m/s in reverse, lateral acceleration below 1 m/s^2, and braking $a_b = 0.6$ m/s^2. The term $0.5|v|$ subtracts the distance covered during the drivetrain lag. Without it the car overshot every stop by about 0.35 m.
4. **Cusp handling.** At the end of a segment the car stops ($|v| < 0.08$ m/s), steers at standstill towards the next segment, and changes gear.
5. **Speed loop.** The PID of Eq. (5) turns v_{ref} (after the reflex layer) into an acceleration command. In Webots it is sent as a cruising speed, or as throttle/brake with gear selection in torque mode.

Controller design and tuning. Pure Pursuit gains: the look-ahead was swept from 1.0 to 3.0 m on test missions. Below 1.4 m the steering oscillated at the cusps, and above 2.2 m the car cut the corners of the turn into the aisle, so we use $L_d = \max(1.6, 0.8|v|)$ (1.2 m minimum in reverse, where the car is slow). For the speed PID, K_p was raised until the rise time was below 1 s. K_i removes the steady error caused by the drivetrain lag, and a small K_d damps the overshoot. The resulting loop has a phase margin of 46° (Section 9.3).

7.5 Reflexive layer

Algorithm 2 lists the reflex rules in priority order. The key quantity is the *swept contact distance* s_c (novelty 2). All range readings are converted into points in the car frame. The car outline, inflated by 0.2 m, is moved along the arc of the current steering angle in 0.1 m steps, and s_c is the arc length at which the first point enters it. The TTC speed limit is then

$$v_{allow} = \max\Big(0,\ \frac{s_c - d_{stop}}{T_{safe}} - v_{obs}\Big), \qquad d_{stop} = 0.45 \text{ m},\ T_{safe} = 2.5 \text{ s}, \tag{7}$$

where v_{obs} is the closing speed of the obstacle itself, estimated from the change of s_c between steps. The speed therefore falls smoothly as the car gets closer, instead of the stop–go behaviour of a fixed-distance rule.

Algorithm 2 Reflex layer (runs every control step, after the procedural layer)

Input: proposed (δ, v_{ref}), ranges $r_1..r_{26}$, speed v, state

if state = DOCK **then** ▷ R3: close-range reactive rules inside the slot
  if $r_{rear} < 0.25$ m **then return** $(\delta, 0)$ and mark the mission complete
  end if
  if both side groups see a car within 1.5 m **then** $\delta \leftarrow \delta + \text{clip}(0.12\,(d_{left} - d_{right}), \pm 0.08)$
  end if
  return (δ, v_{ref})
end if
$s_c \leftarrow$ swept contact distance along the arc of δ in the direction of v_{ref}
if $s_c < d_{stop}$ **then** $v_{ref} \leftarrow 0$ ▷ R1 emergency stop
else $|v_{ref}| \leftarrow \min(|v_{ref}|, v_{allow})$ from Eq. (7) ▷ R2 TTC limit
end if
if held at $v \approx 0$ for more than 4 s **then** ▷ R4
  add a 0.9 m box at the contact point; re-plan (budget 12 000 nodes); keep the old path if no route exists
end if
return (δ, v_{ref})

7.6 Localisation

The raw GPS position is fused with dead reckoning from the wheel speed and the IMU heading by the Kalman filter of Section 5.5 ($R = 0.15^2$ m^2, $Q = 0.03^2$ m^2/s). A 5σ innovation gate rejects outliers. The yaw comes straight from the IMU, so x and y decouple into two scalar filters, which are exact and cost almost nothing.

8. Simulation Setup

8.1 Platforms

Webots R2025a [7] with the vehicle library (`BmwX5` PROTO, Ackermann steering, `Driver` API), the `Pedestrian` PROTO and a Python 3.11 controller. NumPy and SciPy are the only libraries the agent needs. **Headless twin** (`sim/simulator.py`): the same lot, car dimensions and sensor layout, with a kinematic bicycle model plus steering-rate limit and drivetrain lag, ray-cast range sensors, GPS/IMU/odometry noise and scripted pedestrians. The twin runs a mission in 1–5 s, so we could run 525 missions for statistics, which would take days in Webots. Both run the identical `ValetAgent`.

8.2 World and parameters

The Webots world (Fig. 8) is generated by `tools/make_world.py` from the same random scenario (seed) that the twin uses: floor, slot lines, walls, kerb, parked cars (as solid boxes) and pedestrians with a back-and-forth trajectory. The script also writes `lot_config.json`, which the controller reads as the facility message. Table 4 lists the main parameters.

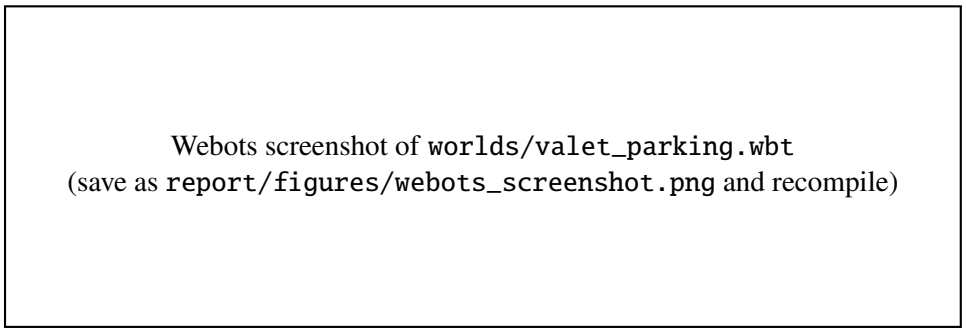


Figure 8: Webots world: BMW X5 valet car, parked cars and a pedestrian (seed 3, 70% occupancy)

Table 4: Simulation parameters

Parameter	**Value**
Webots basic time step / control period	16 ms / 48 ms (twin: 50 ms)
Lot size, slots, aisles	39.6 m × 36 m, 48 slots (2.8 × 5.5 m), 7 m aisles
Occupancy (random per seed)	50, 70, 85, 92%
GPS / heading / range / speed noise (σ)	0.05–0.30 m / 0.5° / 2 cm / 0.02 m/s
Hybrid A* grid, primitives	0.5 m × 5°, 5 steering × 2 directions, 1 m
Costmap resolution, safety margin	0.1 m, 0.2 m
Pedestrians	4, speed 0.9–1.4 m/s, random start delay 0–6 s
Success criterion	PARKED, no contact, inside slot, $\lvert e_{lat}\rvert \le 0.3$ m, $\lvert e_{\theta}\rvert \le 5°$
Seeds per configuration	30 (E1–E4), 15 (E5–E7)

8.3 How to run the code

```
pip install -r requirements.txt
python -m unittest discover -s tests
python experiments/run_experiments.py --n 30
python experiments/make_figures.py
python tools/make_world.py --seed 3     # then open .wbt
```

Options such as `-planner grid`, `-law stanley`, `-reflex fixed` and `-selector nearest` switch to the baseline methods, both in the twin and in the Webots world.

9. Results and Analysis

All results come from the headless twin, with 30 random lots per configuration (15 for E5–E7) and a fixed seed for every mission, so every number can be reproduced. Unless stated otherwise, the agent uses Hybrid A* with the funnel, Pure Pursuit, the TTC reflex, the utility slot choice, the Kalman filter and 0.05 m GPS noise. Values are mean ± standard deviation.

9.1 A complete mission

Figure 9 shows a typical mission. The car leaves the drop-off point, drives forward along aisle 1 past the chosen slot, stops, changes into reverse and backs into the slot, straightening in the funnel. Figure 10 shows the speed, steering and cross-track error over time. The speed follows the profile with a small overshoot. The steering stays below the limit, with a reserve from the 88% rule, and the cross-track error stays within about 10 cm.

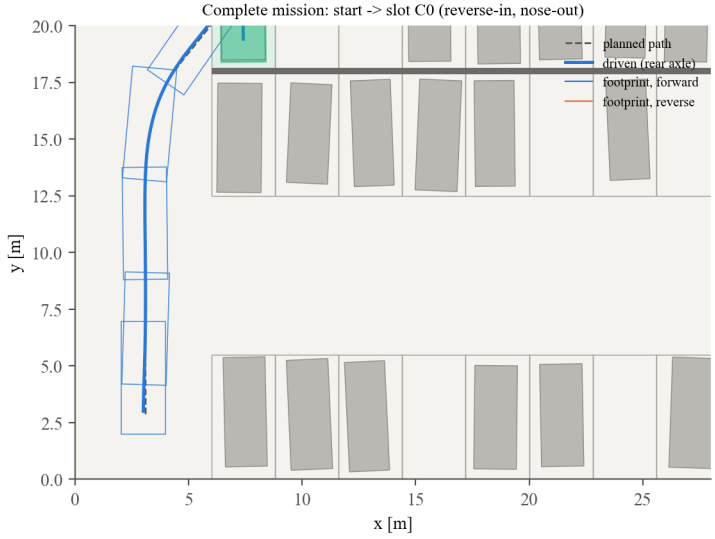


Figure 9: A complete mission: planned path, driven path and car footprints (blue forward, orange reverse)

9.2 E1: Deliberative planners

Table 5 and Fig. 12 compare the planners. Grid A* plans in milliseconds, but its path has a 90° corner into the slot that a car with R_{min} = 4.87 m cannot follow. When the car tried to follow it, the reflex layer prevented every contact (0% collisions), but the car always ended up wedged against a neighbouring car, so its success rate is 0%. This confirms that a planner must

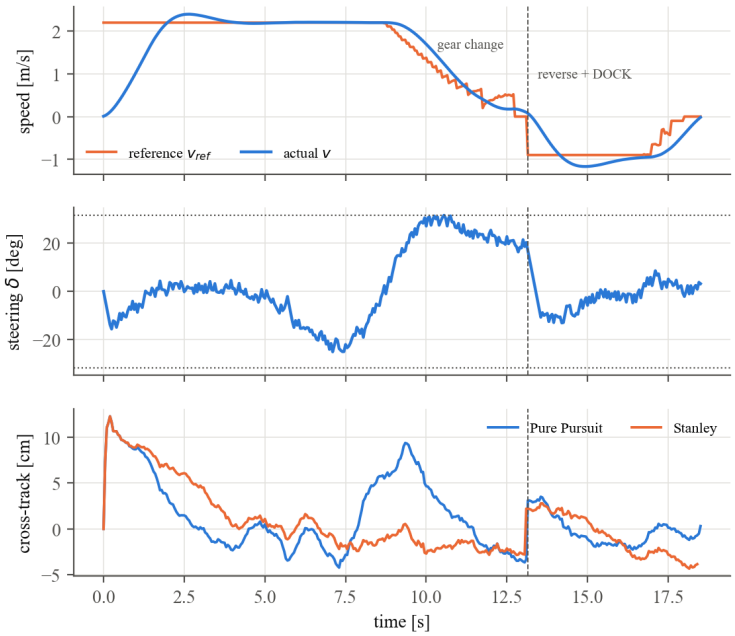


Figure 10: Speed, steering angle and cross-track error during the mission of Fig. 9

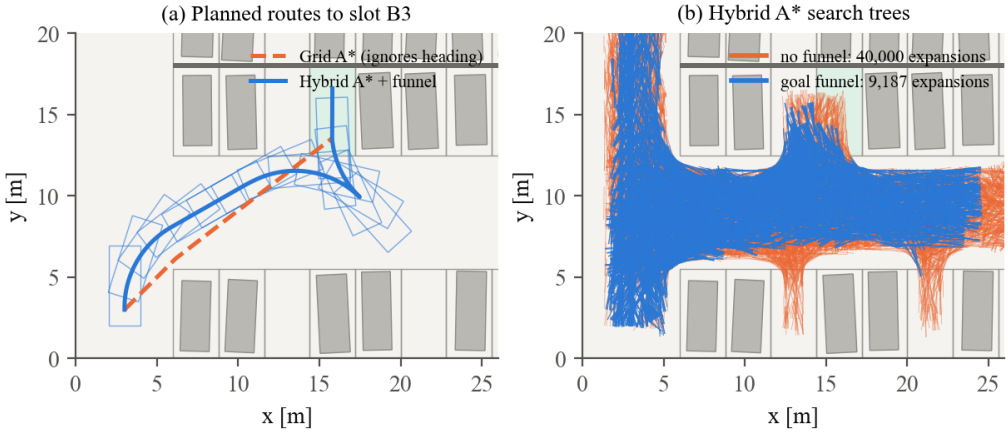


Figure 11: (a) Grid A* and Hybrid A* routes to slot B3. (b) Search trees with and without the goal funnel

respect the vehicle's kinematics. Plain Hybrid A* needs 54,470 expansions on average and often runs out of its 40 000-node budget, because it must hit a small (x, y, θ) goal (Fig. 11b). With the goal funnel, the search needs 5.5× fewer expansions and parks successfully in 100% of the lots. The planning times in Table 5 were measured while seven missions ran in parallel on an 8-core laptop. Run alone, the funnel planner needs 1.65 s on average (median 1.18 s, maximum 3.4 s, 15 lots), so objective O3 is met. Paths are 24.4 m long on average with 1.17 gear changes, which is a single "drive past and reverse in" manoeuvre.

Table 5: E1 — planner comparison (30 lots, 70% occupancy)

Metric	Grid A*	Hybrid A* (no funnel)	Hybrid A* + funnel
Missions (seeds)	30	30	30
Success rate (%)	0.0	80.0	100.0
Collisions (%)	0.0	0.0	0.0
Planning time (s)	0.02 ± 0.02	27.48 ± 24.98	5.08 ± 2.80
Node expansions	1075 ± 1562	54470 ± 49173	9968 ± 5291
Path length (m)	6.8 ± 7.3	27.5 ± 8.5	24.4 ± 6.6
Gear changes	0.00 ± 0.00	1.00 ± 0.00	1.17 ± 0.52
Drive time, successful (s)	–	23.4 ± 5.3	24.1 ± 6.0

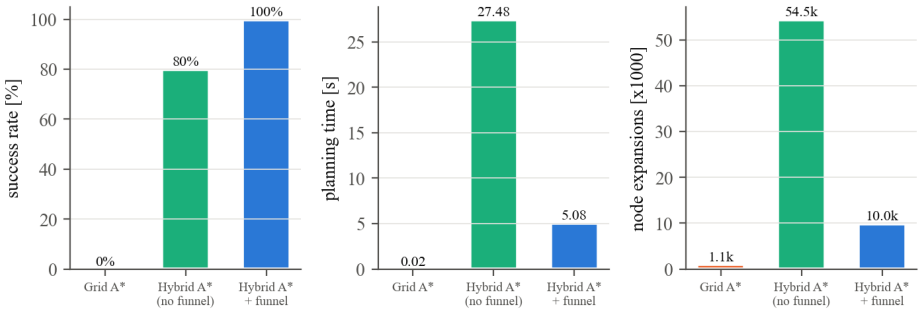


Figure 12: E1 — success rate, planning time and node expansions of the three planners

Table 6: E2 — path-tracking laws (30 lots, Hybrid A* paths)

Metric	Pure Pursuit	Stanley
Success rate (%)	100.0	56.7
Mean $\|e_{ct}\|$ (cm)	3.3 ± 1.8	4.3 ± 1.8
Max $\|e_{ct}\|$ (cm)	11.0 ± 5.5	13.7 ± 4.2
Final |lateral| (cm)	4.2 ± 3.0	15.2 ± 7.4
Final heading error (deg)	1.18 ± 0.86	2.66 ± 1.27
Drive time (s)	24.1 ± 6.0	27.7 ± 9.1

9.3 E2: Steering laws and speed loop

Both laws track well (Table 6, Fig. 10). Pure Pursuit has a mean cross-track error of 3.3 cm and parks with 4.2 cm lateral and 1.2° heading error, so objective O2 is met. Stanley tracks the forward part almost as well (4.3 cm), but in reverse its rear-axle reference makes it react later. Its final errors are larger (15.2 cm, 2.7°), so only 57% of its missions end within the tolerance, against 100% for Pure Pursuit. We therefore use Pure Pursuit by default. A first version of Stanley, which tracked the rear-axle path with the front axle, cut every corner by about 0.8 m. Projecting the path to the front axle fixed this (Section 7.4).

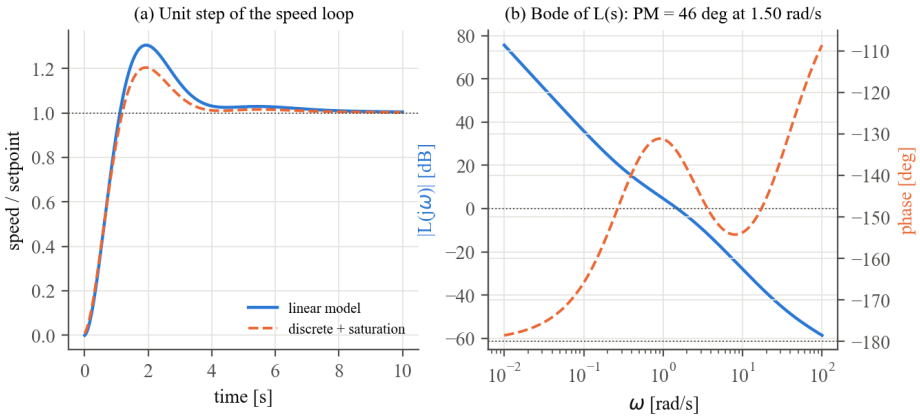


Figure 13: Speed loop: (a) unit-step response, (b) Bode plot of the open loop $L(s) = C(s)G(s)$

Figure 13 analyses the speed loop in the time and frequency domain. The linear model gives a rise time of 0.78 s, 31% overshoot and a 2% settling time of 6.7 s. The discrete loop with actuator saturation gives 0.85 s, 20% and 3.7 s. Saturation limits the acceleration, which in turn limits the integral build-up. The open loop crosses 0 dB at 1.50 rad/s with a phase margin of 46°, so the loop is stable with a reasonable margin. The overshoot is acceptable here because the speed profile ramps v_{ref} rather than stepping it, and the overshoot appears only in the cruise phase, not at the stop.

9.4 E3: Reflexive layer and pedestrian safety

Table 7: E3 — reflex designs with four crossing pedestrians (30 lots)

Metric	No reflex	Fixed-distance	Swept TTC
Success rate (%)	80.0	76.7	100.0
Missions with contact (%)	20.0	16.7	0.0
Min. pedestrian gap (m)	6.31 ± 6.00	6.71 ± 6.13	6.40 ± 5.96
Re-plans per mission	0.00 ± 0.00	5.47 ± 15.81	0.03 ± 0.18
Drive time, successful (s)	21.0 ± 2.6	31.6 ± 40.8	25.8 ± 7.9

Without a reflex layer, the car hit a pedestrian in 20% of the missions (Table 7). The classic fixed-distance rule (stop if the straight corridor ahead is blocked within 2.5 m) reduced this to 17%. Its remaining contacts were side-swipes during turns, where a person beside the front wing is outside the straight corridor. It also stops for parked cars that are near but not in the way, and it caused stop–go driving and long waits. The swept-footprint TTC rule reached 0% contacts and 100% success (objective O4). It also drove more smoothly, because the speed falls gradually (Fig. 14). Developing this rule was iterative. The first version used only the bumper sensors and still had side contacts, so we added six side sensors per side. The second version could deadlock with a waiting pedestrian, so we added rule R4, which re-plans around the obstacle at the sensed contact point.

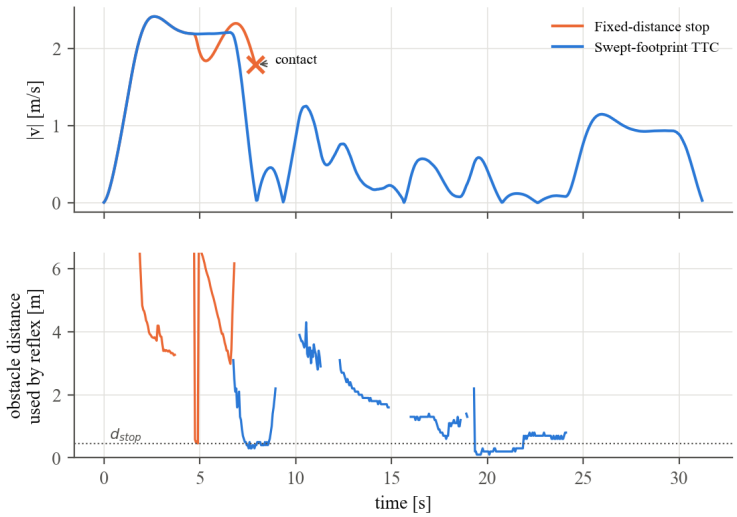


Figure 14: Same lot and pedestrians: speed and obstacle distance seen by the fixed-distance and TTC reflexes

9.5 E4–E7: Agent decisions, noise and robustness

Table 8: E4 — slot-selection policy (30 lots, 85% occupancy)

Metric	Nearest slot	Utility agent
Success rate (%)	96.7	100.0
Valet cycle length (m)	54.9 ± 9.1	50.7 ± 7.5
Parking path length (m)	24.6 ± 5.4	28.6 ± 8.8
Planning time (s)	4.57 ± 1.34	5.55 ± 2.92
Drive time (s)	24.6 ± 5.8	25.6 ± 6.6

E4 slot selection. The utility agent picks a different slot from the nearest-slot rule in 14 of the 30 lots, typically one in the middle rows that is a little further from the drop-off but much closer to the pick-up zone, or one with free neighbours. Over the whole valet cycle (drive in

+ drive out), it saves 7.6% of the distance (Table 8, Fig. 16b) without lowering the success rate.

Table 9: E5 — GPS noise: raw GPS vs Kalman-filtered pose (15 lots per level)

GPS σ (m)	Success raw (%)	Success KF (%)	|lat| raw (cm)	|lat| KF (cm)
0.00	100	100	3.5	3.7
0.05	100	100	4.3	4.2
0.10	87	93	5.8	5.1
0.20	60	100	7.8	4.2
0.30	20	100	14.7	4.4

E5 sensor noise. With raw GPS the controller steers on noise. At $\sigma = 0.2$ m success falls to 60%, and at 0.3 m to 20%, mostly because the final lateral error exceeds the tolerance. With the Kalman filter, success stays at 100% and 100% (Table 9, Fig. 15). This matters for real cars, where GPS in a car park is poor.

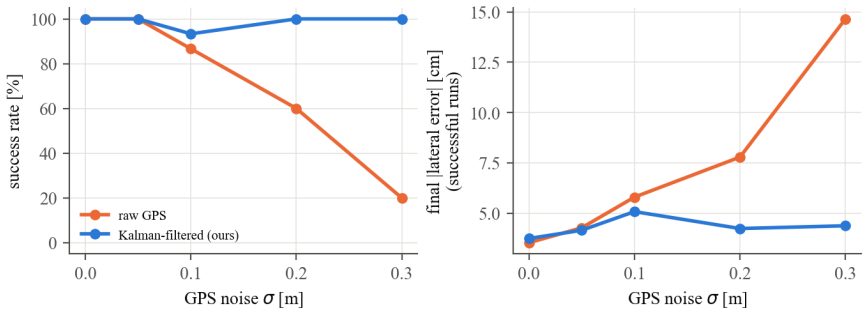


Figure 15: E5 — success rate and final lateral error against GPS noise

Table 10: E6 / E7 — lot occupancy and the "slot taken" event (15 lots each)

Scenario	Runs	Success (%)	Planning time (s)	Drive time (s)
50%	15	100	5.79 ± 5.11	22.4 ± 3.6
70%	15	100	5.27 ± 2.62	23.1 ± 3.7
85%	15	100	5.15 ± 2.34	23.4 ± 4.4
92%	15	93	5.85 ± 2.77	28.7 ± 8.5
70% + slot taken	15	100	9.55 ± 3.20	29.2 ± 7.5

E6 occupancy and E7 dynamic events. Success stays high from 50% to 92% occupancy (Table 10, Fig. 16a). Fuller lots give fewer free slots, and often only harder ones, so planning takes longer. In E7, another car takes the agent's chosen slot 4–8 s into the mission. The facility message moves the state machine back to SELECT, the agent chooses the next-best slot and re-plans from its current pose, with 100% success.

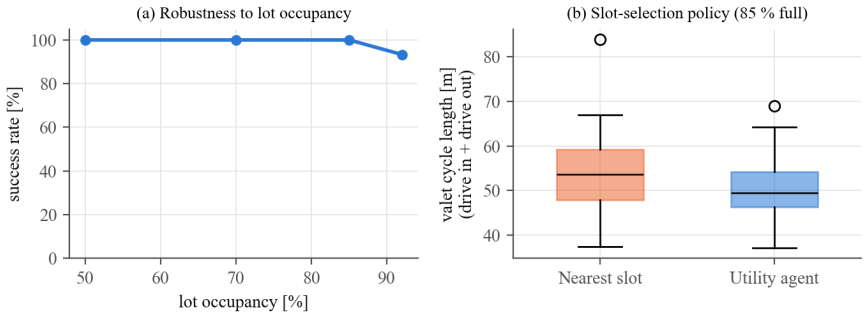


Figure 16: (a) Success rate against lot occupancy (E6). (b) Valet cycle length for the two slot policies (E4)

9.6 Parking accuracy, and what did not work

Over all successful default missions the final lateral error is about 4 cm and the heading error about 1.2° (Tables 5 and 6), well inside the slot, which leaves 0.42 m clearance on each side.

Honest discussion of limits.

- **Things that failed first and how we fixed them.** (i) Planning with the full steering lock left no reserve for feedback, and the car ended up to 10° crooked. Using 88% of the limit fixed it. (ii) The car overshot every stop by 0.35 m until the speed profile accounted for the drivetrain lag. (iii) A funnel that accepted poses deep inside the slot gave too short a straight segment for the heading to settle, so we set a minimum length of 2.5 m.
- **Remaining failures.** The default agent did not fail in E1–E4. The few failures in E5 and E6 (one lot each, at 0.10 m noise and at 92% occupancy) are missions where the final heading error just exceeded 5° or where the only remaining slot needed a long multi-shunt manoeuvre.
- **Twin vs Webots.** The batch statistics come from the kinematic twin. Webots adds tyre slip and suspension, which can make the reverse slightly less accurate. The live Webots demonstration uses the same controller, but its run-to-run statistics would need many more runs than we could do in the project time.
- **Python speed.** Planning takes 1–3.5 s in Python. A C++ implementation, as in industry stacks, would be about 50 times faster.
- **Pedestrian model.** Our pedestrians are scripted and courteous. Real people can be faster or behave unpredictably, which needs a prediction model.

10. Contribution

Table 11: Technical contribution of each member

S. No.	Name and SAP ID	Technical contribution (modules / algorithms / code files / experiments)	Report sections + signature
1	Saurabh Kumar (500124288), team lead	Parking-lot model and random scenario generator (`valet/lot.py`); occupancy grid and distance-transform collision check (`valet/costmap.py`); Webots world generator (`tools/make_world.py`: slots, walls, parked cars, BmwX5 with 26 sensors, pedestrians); Webots controller (`controllers/valet_parking`); live Webots demo; Fig. 4, Fig. 8.	Abstract, 1, 2, 13
2	Naman Chanana (500119485)	Grid A* baseline with octile heuristic and line-of-sight smoothing (`valet/planning/grid_astar.py`); Hybrid A* with Ackermann motion primitives, Dijkstra heuristic and the slot goal funnel (`hybrid_astar.py`, novelty 1); path class and cusp splitting (`path.py`); unit tests; experiment E1, Fig. 11, Fig. 12.	3, 4
3	Anirvan Ranjan (500119330)	Three-layer agent and mission state machine with slot-taken and blocked events (`valet/agent.py`); utility-based slot selection (`slot_selection.py`, novelty 3); GPS–odometry Kalman filter (`localisation.py`); experiments E4, E5, E7; Fig. 3, 7.	5, 6
4	Aditya Pratap Singh Sisodia (500121985)	PID with anti-windup (`control/pid.py`); Pure Pursuit and Stanley for forward and reverse driving, incl. front-axle projection and curvature feed-forward (`control/steering.py`); path tracker with lag-aware speed profile and cusp handling (`control/tracker.py`); gain tuning, Bode/step analysis; experiment E2; Fig. 10, 13; Webots controller test.	7, 8

S. No.	Name and SAP ID	Technical contribution	Sections + signature
5	Priyal Sharma (500122041)	Reflexive layer R1–R4 with the swept-footprint TTC rule (`valet/reflex.py`, novelty 2); headless simulator twin with bicycle model, ray-cast sensors and pedestrians (`sim/`); Monte Carlo runner and figure/table generation (`experiments/`); experiments E3, E6; Fig. 14–16.	9, 12, Appendix

Every claim above can be traced to the commit history of the group repository (Appendix): each file was committed by the member listed above from his or her own account. All members took part in the integration tests, the recorded meetings and the demonstration video.

11. Co-learning Reflection

Saurabh Kumar: On my own I learned how Webots builds a vehicle from the `Car` PROTO and how the `Driver` API differs from ordinary motor control, for example that the steering angle is positive to the right. Generating the world from Python instead of drawing it by hand was my idea, and it saved us when the lot layout changed three times. From Naman I learned how Hybrid A* uses motion primitives, and from Priyal how a distance transform makes collision checks fast. I taught the team the Webots workflow and how to read the controller console when the car misbehaved.

Naman Chanana: I studied A*, Reeds–Shepp curves and the Hybrid A* paper on my own. The hardest part was seeing why the search was slow, and drawing the search tree (Fig. 11b) showed that it wasted most nodes near the slot, which led to the goal-funnel idea. From Aditya I learned why the planner must leave a steering reserve for the controller, and from Anirvan how a state machine handles failures. I explained admissible heuristics and the cost function to the team before the viva practice.

Anirvan Ranjan: I learned the agent types of Russell and Norvig and the three-layer architecture by reading on my own, and then turned them into one `step()` function that every layer plugs into. Writing the utility function taught me that the “best” slot depends on the whole valet cycle, not only on the drive in. From Priyal I learned how to design a fair experiment with fixed seeds, and from Aditya the basics of a Kalman filter. I taught the others how the layers pass commands and events, so that each of us could debug our own module inside the full agent.

Aditya Pratap Singh Sisodia: On my own I learned Pure Pursuit and Stanley, and why reversing flips the sign of the steering. I found out the hard way that Stanley must track the

front-axle path, after it cut every corner by 0.8 m. I also learned to read a Bode plot for a real loop and to explain overshoot from the drivetrain lag. From Saurabh I learned how Webots runs a controller, and from Naman how the planner's cusps define my segments. I taught the team how to tune a PID and how to read the tracking plots.

Priyal Sharma: I learned ray casting, time to collision and Monte Carlo evaluation on my own. The most useful lesson was that a good-looking demo can hide failures: the first pedestrian tests showed side-swipes that we only found by running hundreds of missions and checking every contact. From Anirvan I learned the agent's event handling, which I used for the blocked-replan rule, and from Saurabh the sensor modelling in Webots. I taught the team how to run the experiment scripts and regenerate all figures with one command.

12. Conclusion and Future Scope

Main findings. We built a complete autonomous valet parking agent, organised as a declarative–procedural–reflexive hierarchy, that runs in Webots and in a headless twin. Over 525 simulated missions:

1. Kinematic planning is essential. Grid A* succeeded in 0% of the lots and Hybrid A* in 100%, and the goal funnel made Hybrid A* 5.4× faster.
2. Pure Pursuit with lag-aware speed control parks within a few centimetres and about 1°.
3. The swept-footprint TTC reflex reduced pedestrian contacts to 0% (fixed-distance rule: 17%) and still completed 100% of missions.
4. A utility-based agent shortens the valet cycle by 7.6%, and a simple Kalman filter makes the system robust to 0.3 m GPS noise.

Industrial use. The architecture matches infrastructure-supported AVP as standardised in ISO 23374 [1]. The facility provides the free slots and parked-car outlines, and the vehicle plans and executes the manoeuvre with its own safety layer. The same agent can park cars in factory yards and ports, return rental cars, or move a robotaxi to a charger.

Moving to a real robot. (1) Replace the GPS by indoor localisation (lidar/camera map matching or facility beacons), keeping the Kalman filter interface. (2) Replace the range sensors by the car's ultrasonic sensors or a lidar, which only changes the conversion to points in `reflex.py`. (3) Port the planner to C++ for 50–100× speed. (4) Add a dynamic vehicle model and tyre slip to the controller. (5) Start on a 1:10 scale car (e.g. F1TENTH) before a full-size vehicle.

Future scope. Parallel and angled parking; analytic Reeds–Shepp expansion inside Hybrid A*; prediction of pedestrian motion; several valet cars sharing the lot with reservation of aisles; and retrieval missions from the slot to the pick-up zone.

References

[1] *Intelligent Transport Systems — Automated Valet Parking Systems (AVPS) — Part 1: System Framework, Requirements for Automated Driving and for Communications Interface*, ISO 23374-1:2023, International Organization for Standardization, Geneva, Switzerland, 2023.

[2] Robert Bosch GmbH, "Bosch and Mercedes-Benz: driverless parking approved by authorities," press release, Stuttgart, Germany, 2022.

[3] *Taxonomy and Definitions for Terms Related to Driving Automation Systems for On-Road Motor Vehicles*, SAE Standard J3016_202104, SAE International, 2021.

[4] D. Dolgov, S. Thrun, M. Montemerlo, and J. Diebel, "Path planning for autonomous vehicles in unknown semi-structured environments," *International Journal of Robotics Research*, vol. 29, no. 5, pp. 485–501, 2010.

[5] R. A. Brooks, "A robust layered control system for a mobile robot," *IEEE Journal on Robotics and Automation*, vol. 2, no. 1, pp. 14–23, 1986.

[6] E. Gat, "On three-layer architectures," in *Artificial Intelligence and Mobile Robots*, D. Kortenkamp, R. P. Bonasso, and R. Murphy, Eds. Menlo Park, CA, USA: AAAI Press, 1998, pp. 195–210.

[7] O. Michel, "Webots: Professional mobile robot simulation," *International Journal of Advanced Robotic Systems*, vol. 1, no. 1, pp. 39–42, 2004.

[8] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[9] J. A. Reeds and L. A. Shepp, "Optimal paths for a car that goes both forwards and backwards," *Pacific Journal of Mathematics*, vol. 145, no. 2, pp. 367–393, 1990.

[10] B. Paden, M. Čáp, S. Z. Yong, D. Yershov, and E. Frazzoli, "A survey of motion planning and control techniques for self-driving urban vehicles," *IEEE Transactions on Intelligent Vehicles*, vol. 1, no. 1, pp. 33–55, 2016.

[11] R. C. Coulter, "Implementation of the pure pursuit path tracking algorithm," Robotics Institute, Carnegie Mellon University, Pittsburgh, PA, USA, Tech. Rep. CMU-RI-TR-92-01, 1992.

[12] S. Thrun *et al.*, "Stanley: The robot that won the DARPA Grand Challenge," *Journal of Field Robotics*, vol. 23, no. 9, pp. 661–692, 2006.

[13] J. M. Snider, "Automatic steering methods for autonomous automobile path tracking,"

Robotics Institute, Carnegie Mellon University, Pittsburgh, PA, USA, Tech. Rep. CMU-RI-TR-09-08, 2009.

[14] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2020.

[15] J. C. Hayward, "Near-miss determination through use of a scale of danger," *Highway Research Record*, no. 384, 1972.

[16] R. Rajamani, *Vehicle Dynamics and Control*, 2nd ed. New York, NY, USA: Springer, 2012.

[17] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA, USA: MIT Press, 2005.

Appendix: Links and Logs

GitHub repository: `https://github.com/<team-lead>/g14-valet-parking`
YouTube video (Unlisted): `https://youtu.be/<video-id>`

Repository contents. `valet/` (agent: planners, controllers, reflex layer, slot selection, localisation), `sim/` (headless twin), `controllers/valet_parking/` (Webots controller), `tools/make_world.py` (Webots world generator), `experiments/` (E1–E7 and figures), `tests/` (16 unit and integration tests), `results/` (raw CSV of all missions, figures, tables).

Meeting log

No.	Date	Duration	Attendees	Decisions
1	15/10/2026			Team lead chosen; work split into five modules; GitHub repository created
2	17/10/2026			Hybrid A* chosen over grid A*; three-layer interfaces agreed; reverse-in parking policy
3	20/10/2026			First full mission in the twin; stopping overshoot found and fixed
4	22/10/2026			Side-swipe contacts found; swept-footprint TTC rule and side sensors added
5	24/10/2026			Experiments E1–E7 run; report writing split (Writing Allocation)
6	26/10/2026			Webots demo checked; slides reviewed; timed rehearsal
7	27/10/2026	20 min	All 5	Recorded online presentation with live demo

Peer-assessment sheet (one sheet per member; each member rates every team-mate, 1–5)

Assessor name and SAP ID: _____

No.	Team-mate name and SAP ID	Contribution (1-5)	Reliability / on time (1-5)	Helped others learn (1-5)	Quality of work (1-5)
1					
2					
3					
4					

Signature: _____ Date: _____ *(one sheet per member)*